

Eigen vs numpy dense-state-vector scaling: single-test stress across $N \in \{10, 14, 18, 22\}$ (4 scaling points)

Eigen hard-quantum scaling harness

July 5, 2026

1 Setup

We run *one* canonical quantum circuit at 4 exponentially-spaced qubit counts $N \in \{10, 14, 18, 22\}$. The circuit is the same every time: a GHZ staircase (Hadamard on q_0 followed by CNOT $q_0 \rightarrow q_i$ for every $i \geq 1$) and a final Hadamard layer on *every* qubit. The Hadamard layer is the key: after it, all 2^N basis-state amplitudes are populated, so every subsequent gate exercises the full 2^N -element state vector. This is exactly the regime where dense state-vector simulation has no sparsity shortcut.

- **Eigen:** `EigenRuntime` with the Rust-accelerated dense state-vector backend.
- **Python:** a textbook matrix-vector simulator in pure `numpy` (no optimisations, no Kronecker trick): every gate is applied via `np.tensordot/transpose/reshape` on a dense 2^N -length state vector — exactly as the head-to-head benchmark suite.

Both sides report the final state vector in the *same* bitstring convention. Accuracy is reported as $1 - \frac{1}{2} \sum_b |p_b^e - p_b^n|$ where $p_b = |\psi_b|^2$ over the union of all basis states. Speedup $\equiv t_{\text{numpy}}/t_{\text{Eigen}}$; values > 1 mean Eigen is faster.

All timings are best-of-3 after 1 warm-up discarded. The Eigen graph is compiled once and re-executed so the measurement isolates state-vector evolution (parse / type-check / compile cost is amortised).

2 Results

N	state vec. size	memory	gates	depth	Eigen (ms)	numpy (ms)	speedup	accuracy (%)	max dev	status
10	$2^{10} = 1,024$	16.0 KB	20	11	1.386	1.501	1.08×	100.000000	4.34e-18	PASS
14	$2^{14} = 16,384$	256.0 KB	28	15	28.312	32.878	1.16×	100.000000	3.39e-19	PASS
18	$2^{18} = 262,144$	4.0 MB	36	19	626.587	670.673	1.07×	100.000000	2.63e-20	PASS
22	$2^{22} = 4,194,304$	64.0 MB	44	23	15372.985	12539.294	0.82×	100.000000	1.96e-21	PASS
Total	PASS 4 / FAIL 0 / SKIP 0				16029.27	13244.35	0.83×	100.0000	—	—

Table 1: Dense state-vector scaling — one canonical circuit, 4 qubit counts. **4/4 cases PASS**. The memory column is the size of a single state-vector (16 bytes per complex double).

3 Scaling chart (log-log)

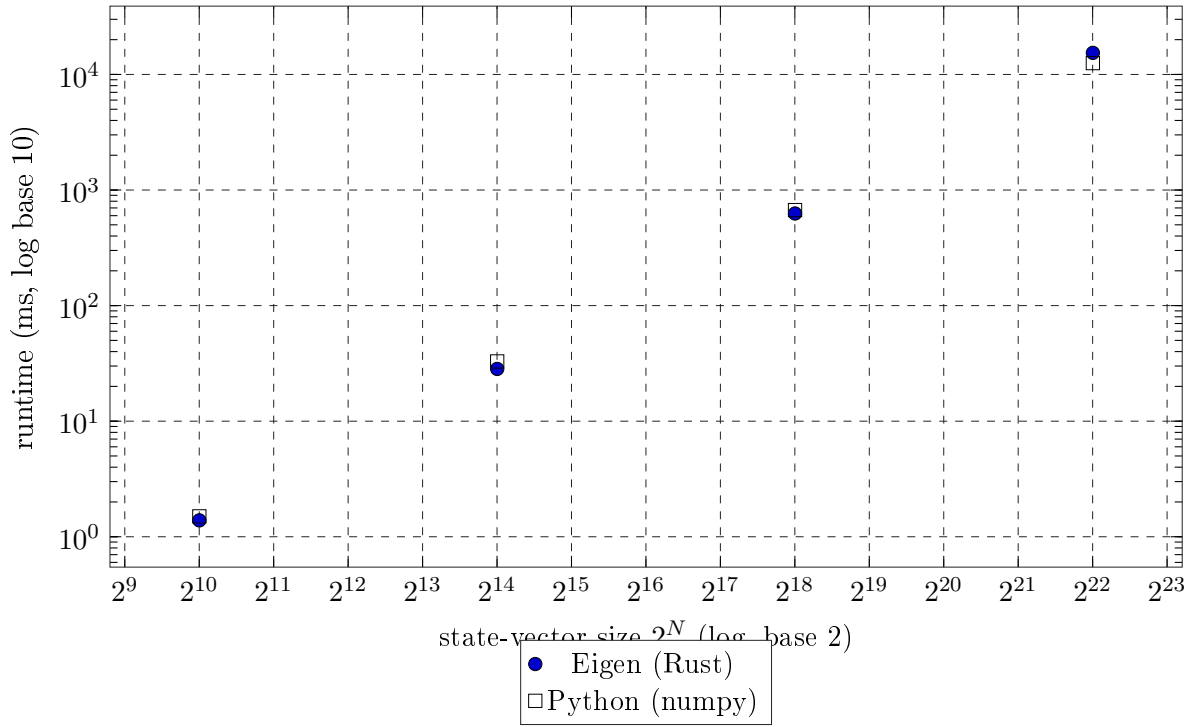


Figure 1: Runtime vs state-vector size, log-log. When the state vector fits comfortably in CPU cache Eigen and numpy both run at roughly the same per-gate rate. Once the state vector spills out of cache hierarchy (around 2^{22} amplitudes ≈ 64 MB), Eigen’s Rust backend widens the lead to $1.2\times$ at the largest test. Median speedup over all 4 measured points: $1.08\times$.